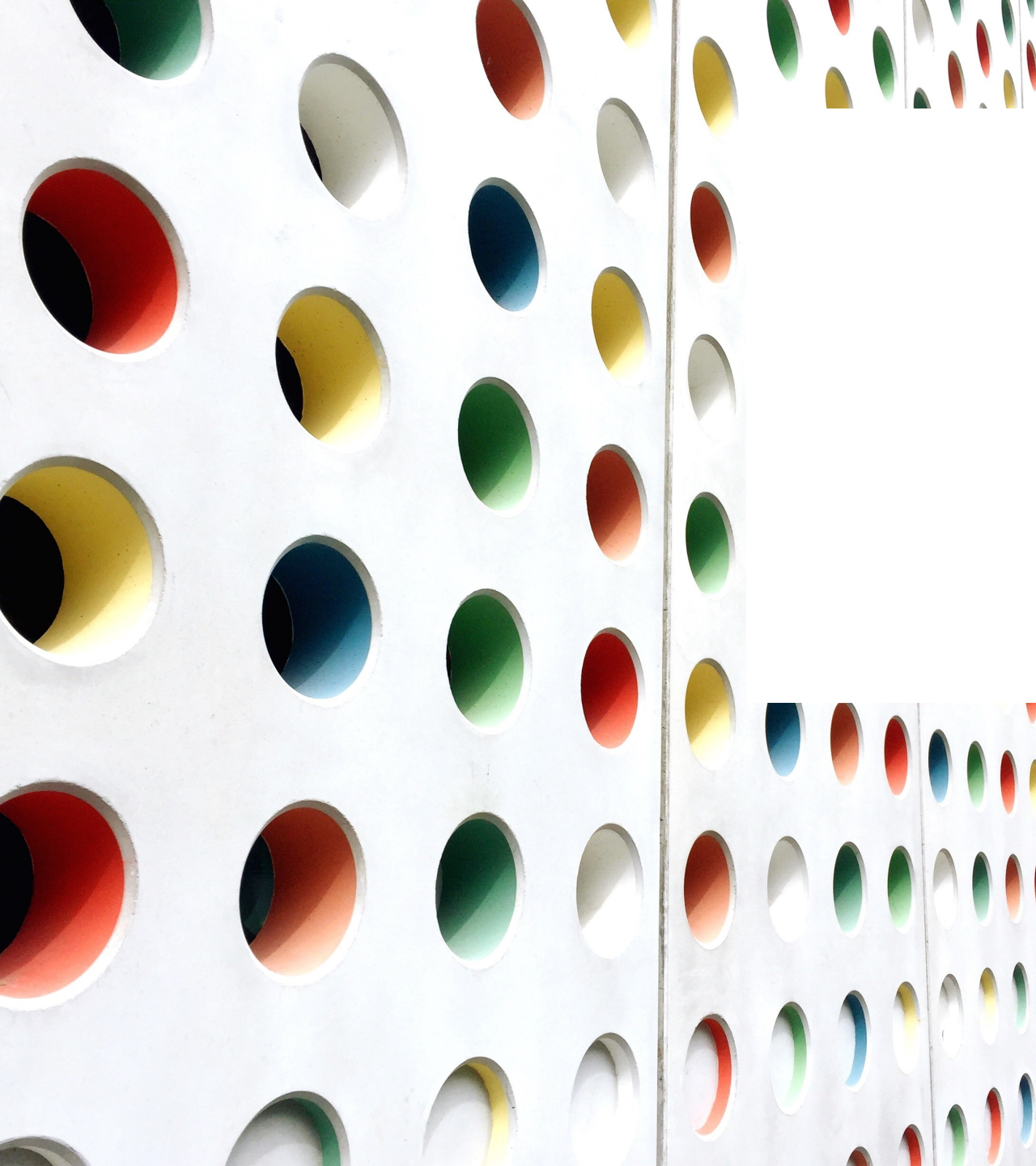




ISP550 INFORMATION SYSTEMS ENGINEERING

**TOPIC 1: INTRODUCTION TO
INFORMATION SYSTEMS ENGINEERING**

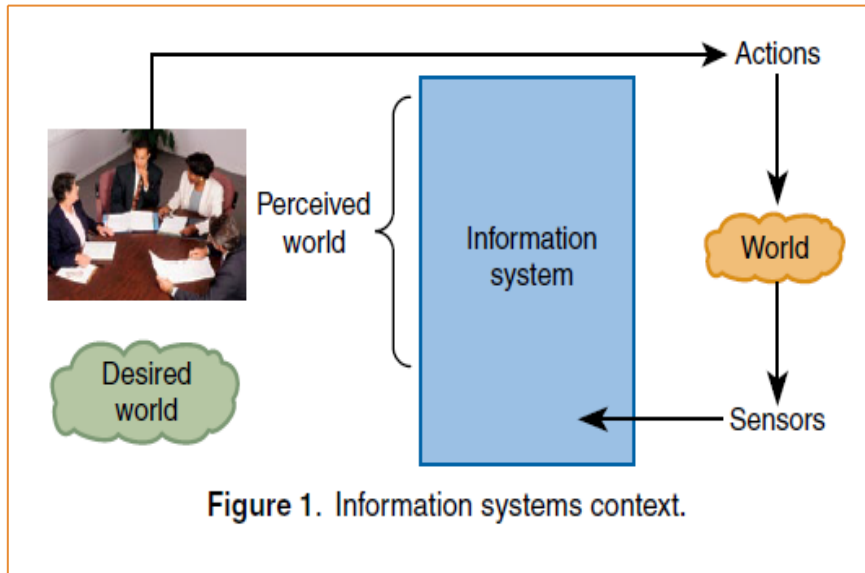
March 2026 – August 2026

Learning Objectives

- Define **Information Systems Engineering**
- Describe the **systems development life cycle** (SDLC)
- Describe the system development **methodologies** such as agile methodologies, eXtreme Programming, Scrum and RUP
- Describe the roles of an **information system engineer**

Definition

- **Definition :** Information systems are computer-based infrastructures, organizations, personnel, and components that **collect, process, store, transmit, display, disseminate, and act on information.**
-

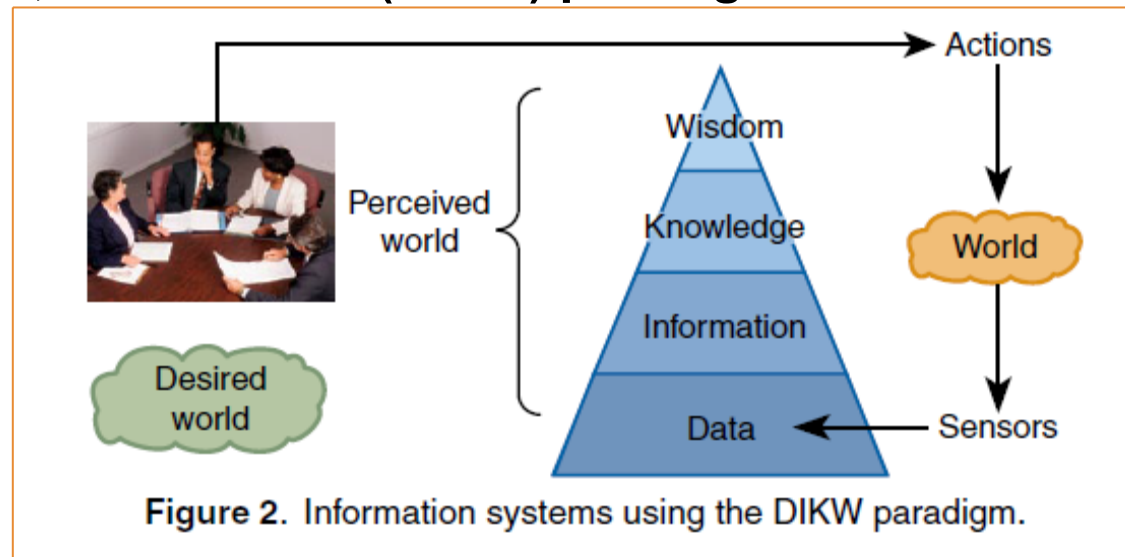


- **Information systems** generally provide computer-based assistance to people engaging their environment as illustrated in Figure 1, where engagements and environments are often too complex and dynamic to be handled manually.

Definition

Complex, dynamic engagements and environments require people to analyze and draw conclusions from an abstracted representation of the world, which enables them to make discrete decisions to achieve a desired effect in the world commensurate with their roles, tasks and capabilities.

The abstraction is sometimes portrayed as a hierarchy (Figure 2) known as the Data, Information, Knowledge, and Wisdom (DIKW) paradigm.



Ackoff, R. L., "From Data to Wisdom," *J. Appl. Syst. Anal.* **16**, 3–9 (1989).

Cleveland, H., "Information as Resource," *The Futurist*, 34–39 (Dec 1982).

Definition

Data

- *are the smallest* symbolic units that describe measured or estimated phenomena.
 - i.e.: sensors produce large volumes of data, but very little is understood by humans.
-

Information

- information is a more abstract understanding of the data derived by fusing data
- lowest layer where the symbolic units are interpretable by humans.

Knowledge

- symbols are sufficiently abstract to enable people to make decisions about and interact with their environments.
- combination of information with necessary data and judgment of historical patterns (experience).

Wisdom

- *is knowledge combined with* insights and common sense.
- It is typically achieved by humans based on knowledge, information, and data.
- hard to derive automatically from lower-level information representations.



TERM INFORMATION SYSTEMS ENGINEERING

CONFERENCE ON ADVANCED INFORMATION SYSTEMS ENGINEERING

(ISE COMMUNITY)

Information systems engineering is situated at the intersection of information systems, databases and software engineering. The term was coined around 1990, possibly in connection with the start-up of the later so successful CAiSE (Conference on Advanced Information Systems Engineering) series of conferences.

It is notable that there are indeed three research communities corresponding to each of the above-mentioned fields, the ISworld, DBworld and SEworld communities.

INTERNATIONAL COUNCIL ON SYSTEMS ENGINEERING

(INCOSE)

Information Systems Engineering is an interdisciplinary approach and means to enable the realization of successful information systems.

ISE focuses on defining customer needs and required functionality early in the development cycle, documenting requirements, then proceeding with design synthesis, and system validation and deployment while considering the complete problem: - Operations & Maintenance, Performance, Cost & Schedule, Test, Realization, Training & Support, Disposal, Social aspects

Information Systems Engineering: What Is It? (Wangler and Backlund, 2005)

TERM INFORMATION SYSTEMS ENGINEERING

- *Information Systems Engineering integrates all the disciplines and specialty groups into a team effort forming a structured development process that proceeds from concept to production to operation.*
- *Information Systems Engineering considers both the business and the technical needs of all customers with the goal of providing a quality product that meets the user needs.*

*There are at least **three broad areas** of knowledge that are needed for successful work in information systems engineering (cf. Iivari [19]):*

- *knowledge of information systems domains and applications,*
- *knowledge about methods, models, and tools for business and systems*
- *analysis and design, deployment, and operations and maintenance,*
- *knowledge of the technology needed for building systems and integrating them with legacy components.*

Information Systems Engineering: What Is It? (Wa Backlund, 2005)

Developing Information Systems and the Systems Development Life Cycle

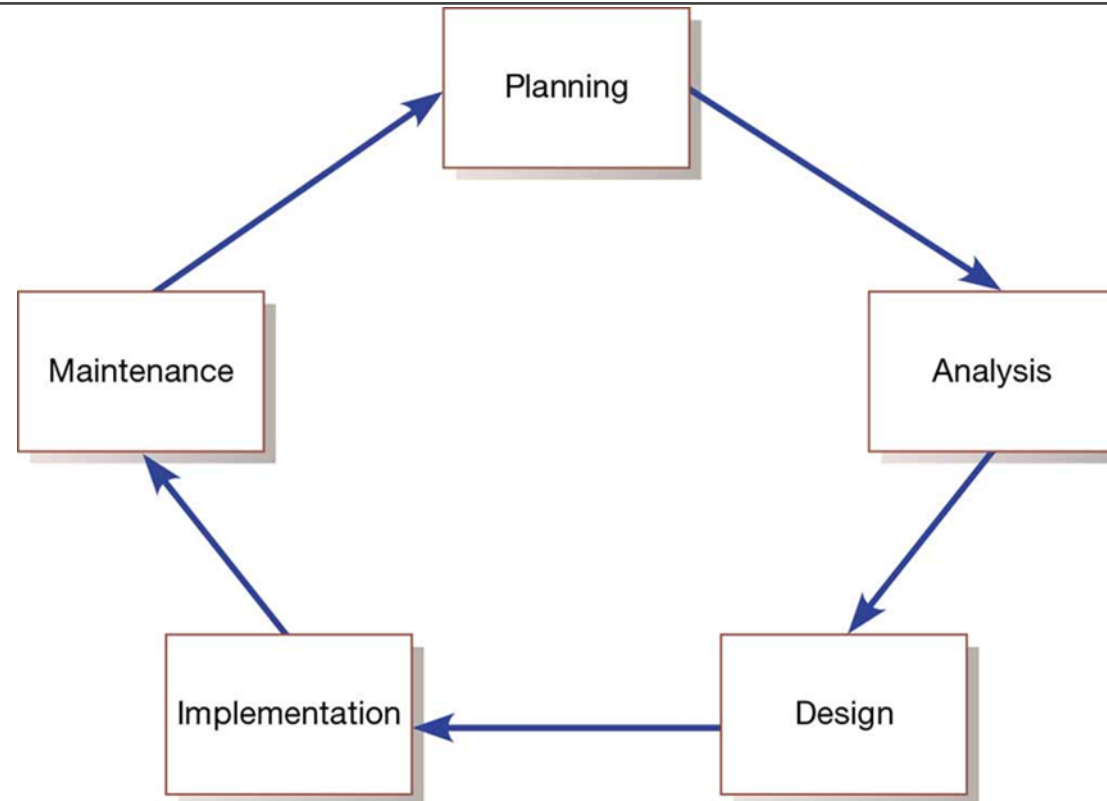
Systems development methodology

- A **standard process** followed in an organization to conduct all the steps necessary to analyze, design, implement, and maintain information systems

The systems development life cycle (SDLC)

- The **traditional methodology** used to develop, maintain, and replace information systems
- Features several phases that mark the progress of the systems analysis and design efforts

Figure 3: Systems Development Life Cycle



Phases of the SDLC [1 of 3]

Planning

- Need for a new or enhanced system is identified
- Needs are identified, analyzed, prioritized, and arranged
- Determine the scope of the proposed system
- Baseline project plan is developed

Analysis

- System requirements are studied from user input and structured
- Requires careful study of current systems, manual and computerized, that might be replaced or be enhanced
- Output is description of the alternate solution recommend by the analysis team

Phases of the SDLC [2 of 3]

Design

- The analyst converts the alternate solution into logical and physical specifications
- *Logical Design*
 - The design process part that is independent of any specific hardware or software platform
- *Physical Design*
 - The logical specifications of the system from logical design are transformed into technology-specific details from which all programming/system construction can be accomplished
 - Choices of language, database, and platform are many times already decided by the organization or client

Phases of the SDLC [3 of 3]

Implementation

- Occurs when the information system is coded, tested, installed, and supported in the organization
- New systems become part of the daily activities of the organization

Maintenance

- The phase in which an information system is systematically repaired and improved
- Organization's needs may change over time requiring changes to the system based on user's needs

Products of SDLC Phases

Phase	Products, Outputs, or Deliverables
Planning	• Priorities for system and projects; an architecture for data, networks, and selection hardware, and information systems management are the result of associated systems • Detailed steps, or work plan, for project • Specification of system scope and planning and high-level system requirements or features • Assignment of team members and other resources • System justification or business case
Analysis	• Description of current system and where problems or opportunities exist, with a general recommendation on how to fix, enhance, or replace current system • Explanation of alternative systems and justification for chosen alternative
Design	• Functional, detailed specifications of system elements (data, processes, inputs, and outputs) • Technical, detailed specifications of all system elements (programs, files, network, system software, etc.) • Acquisition plan for new technology
Implementation	• Code, documentation, training procedures, and support capabilities
Maintenance	• New versions or releases of software with associated updates to documentation, training, and support

Heart of Systems Development

Current practice combines *analysis, design, and implementation* into a single process

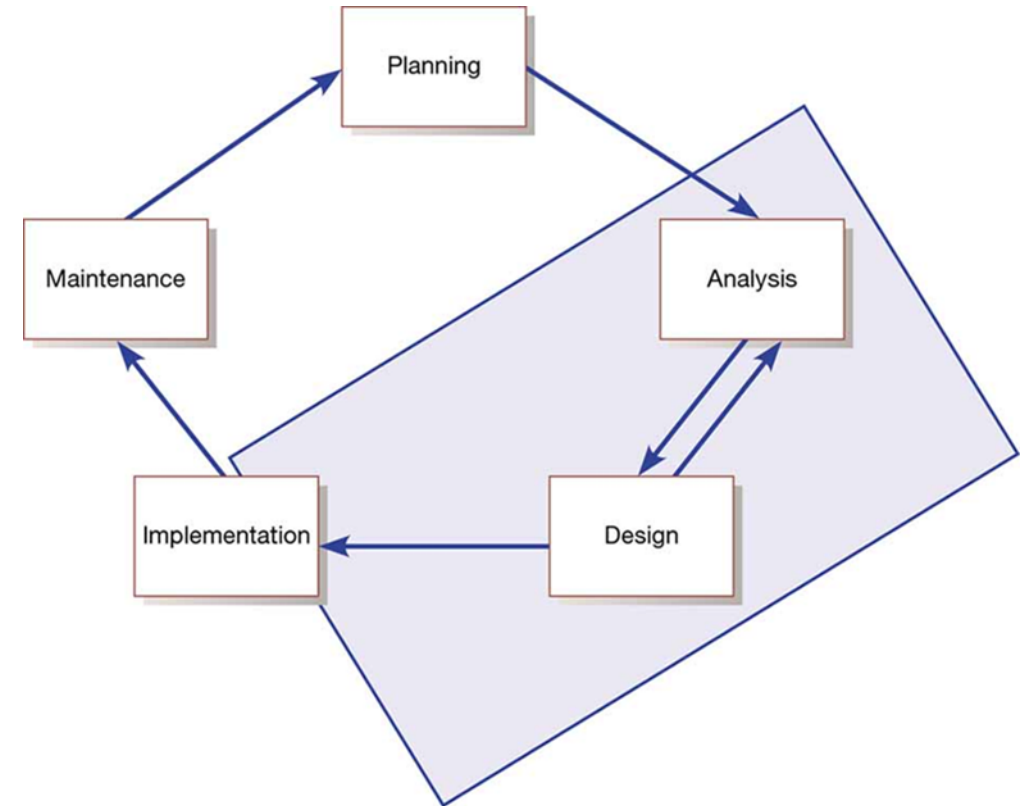


Figure 1-7: Heart of Systems Development

The System Development Life Cycle [SDLC]

There are two general approaches to the SDLC

Predictive Approach

- Waterfall model
- Assumes the project can be **planned** and that the information system **can be developed** according to the plan
- Requirements are **well understood** and/or **low technical risk**

Adaptive Approach to the SDLC

- **Iterative model**
- Assumes the project must be more flexible and adapt to **changing needs** as the project progresses
- Requirements and needs are **uncertain** and/or **high technical risk**

The SDLC Traditional Waterfall Problems

- Once one phase ends another begins, going downhill until complete
- Makes it difficult to go back
- Results in great expense to make changes
- Role of system users or customers narrowly defined
- Focused on deadlines

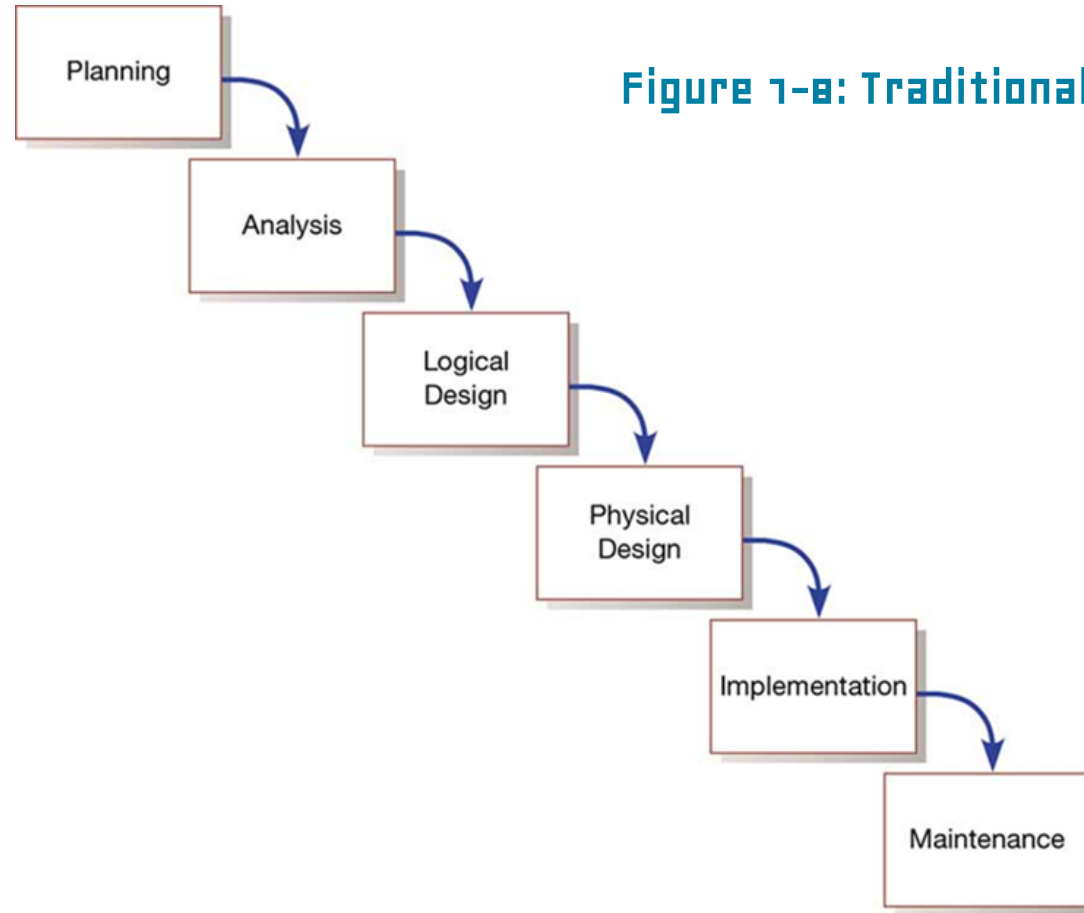


Figure 1-8: Traditional Waterfall SDLC

Agile Methodologies

Agile methodologies share three **key principles**:

1. A focus on adaptive rather than predictive methodologies
2. A focus on people rather than roles
3. A focus on self-adaptive processes

Table 1-2: The Agile Manifesto [1 of 3]

The agile methodologies group argues that software development methodologies adapted from engineering generally do not fit with real-world software development

The Manifesto for Agile Software Development (Table 1-2)

- Seventeen anarchists agree
- We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:
 - Individuals and interactions over processes and tools
 - Working software over comprehensive documentation
 - Customer collaboration over contract negotiation
 - Responding to change over following a plan

Table 1-2: The Agile Manifesto [2 of 3]

- That is, while we value the items on the right, we value the items on the left more. We follow the following principles: (12 Agile Principle)
 1. The highest priority is to **satisfy the customer through early and continuous delivery** of valuable software.
 2. **Welcome changing requirements**, even late in development. Agile processes harness change for the customer's competitive advantage.
 3. **Deliver working software frequently**, from a couple of weeks to a couple of months, with a preference to the shorter timescale.
 4. **Businesspeople and developers work together daily** throughout the project.
 5. **Build projects around motivated individuals**. Give them the environment and support they need and trust them to get the job done.
 6. The most efficient and effective method of conveying information to and within a development team is **face-to-face conversation**.
 7. **Working software** is the primary measure of progress.

Table 1-2: The Agile Manifesto [3 of 3]

8. **Continuous attention** to technical excellence and good design enhances agility.
9. Agile processes **promote sustainable development**. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
10. **Simplicity**—the art of maximizing the amount of work not done—is essential.
11. The best architectures, requirements, and designs emerge from **self-organizing teams**.
12. At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its **behaviour** accordingly.

--Kent Beck, Mike Beedle, Are van Bennekum, Alistair Cockburn, Ward Cunningham, Martin Fowler, James Grenning, Jim Highsmith, Andrew Hunt, Ron Jefferies, Jon Kern, Brian Marick, Robert C. Martin, Steve Mellor, Ken Schwaber, Jeff Sutherland, Dave Thomas (www.AgileAlliance.org)

Agile Methodologies—Not for Every Project

Agile methodologies are **not for everyone**

Fowler recommends an agile process if your project involves

- unpredictable or dynamic requirements
- responsible and motivated developers
- customers who understand the process and will get involved

Table 1-3: Five Critical Factors that Distinguish Agile and Traditional Approaches to System Development

Factor	Agile Methods	Traditional Methods
Size	Well matched to small products and teams Reliance on tacit knowledge limits scalability	Methods evolved to handle large products and teams Hard to tailor down to small products
Criticality	Untested on safety-critical products Potential difficulties with simple design and lack of documentation	Methods evolved to handle highly critical products Hard to tailor down to products that are not critical.
Dynamism	Simple design and continuous refactoring are excellent for highly dynamic environments but a source of potentially expensive rework for highly stable environments	Detailed plans and Big Design Up Front, excellent for highly stable environment but a source of expensive rework for highly dynamic environments
Personnel	Requires continuous presence of a critical mass of scarce experts Risky to use non-agile people	Needs a critical mass of scarce experts during project definition but can work with fewer later in the project, unless the environment is highly dynamic
Culture	Thrives in a culture where people feel comfortable and empowered by having	Thrives in a culture where people feel comfortable and empowered by having their

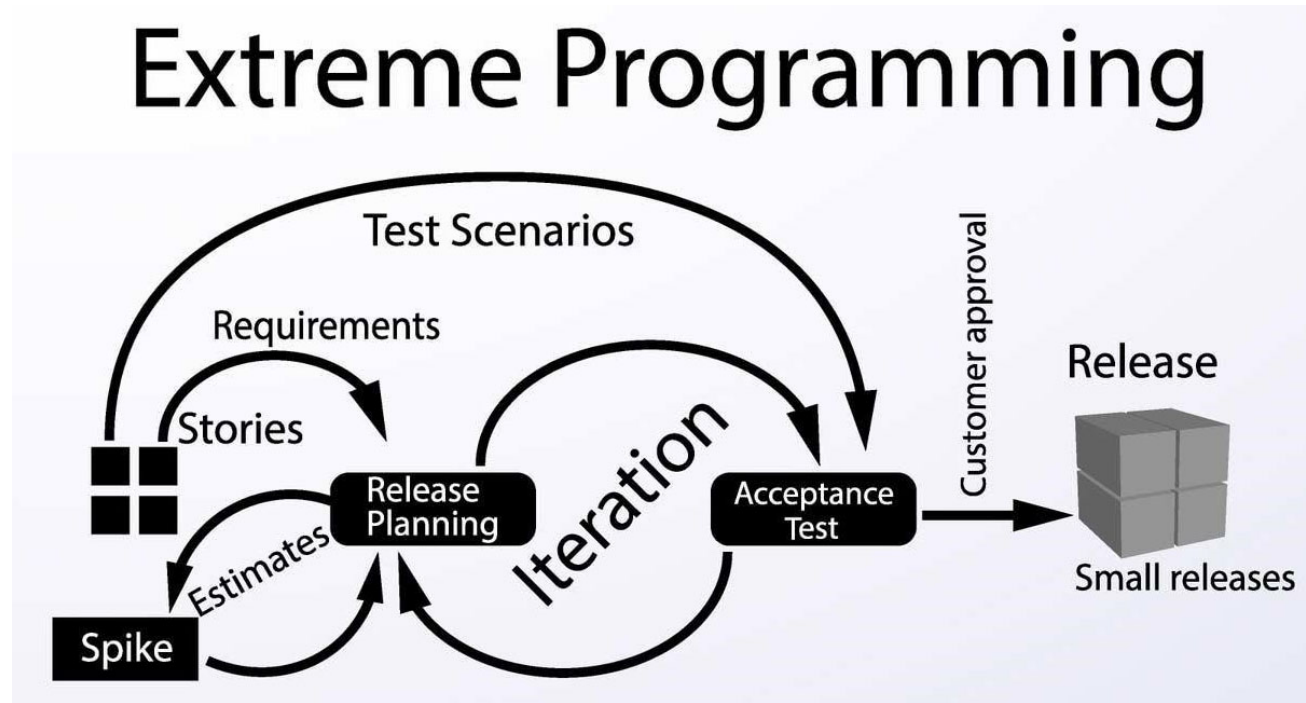
Agile in Practice

Three primary factors critical for success

- **Delivery strategy:** Continuous delivery of working software in short time scales
- **Following agile software engineering practices**
- **Team capability:** Agile principle of building projects around motivated individuals

Agile development offers managers and programmers more choice in their efforts to produce good systems that come in on time and under budget

Figure 8: eXtreme Programming



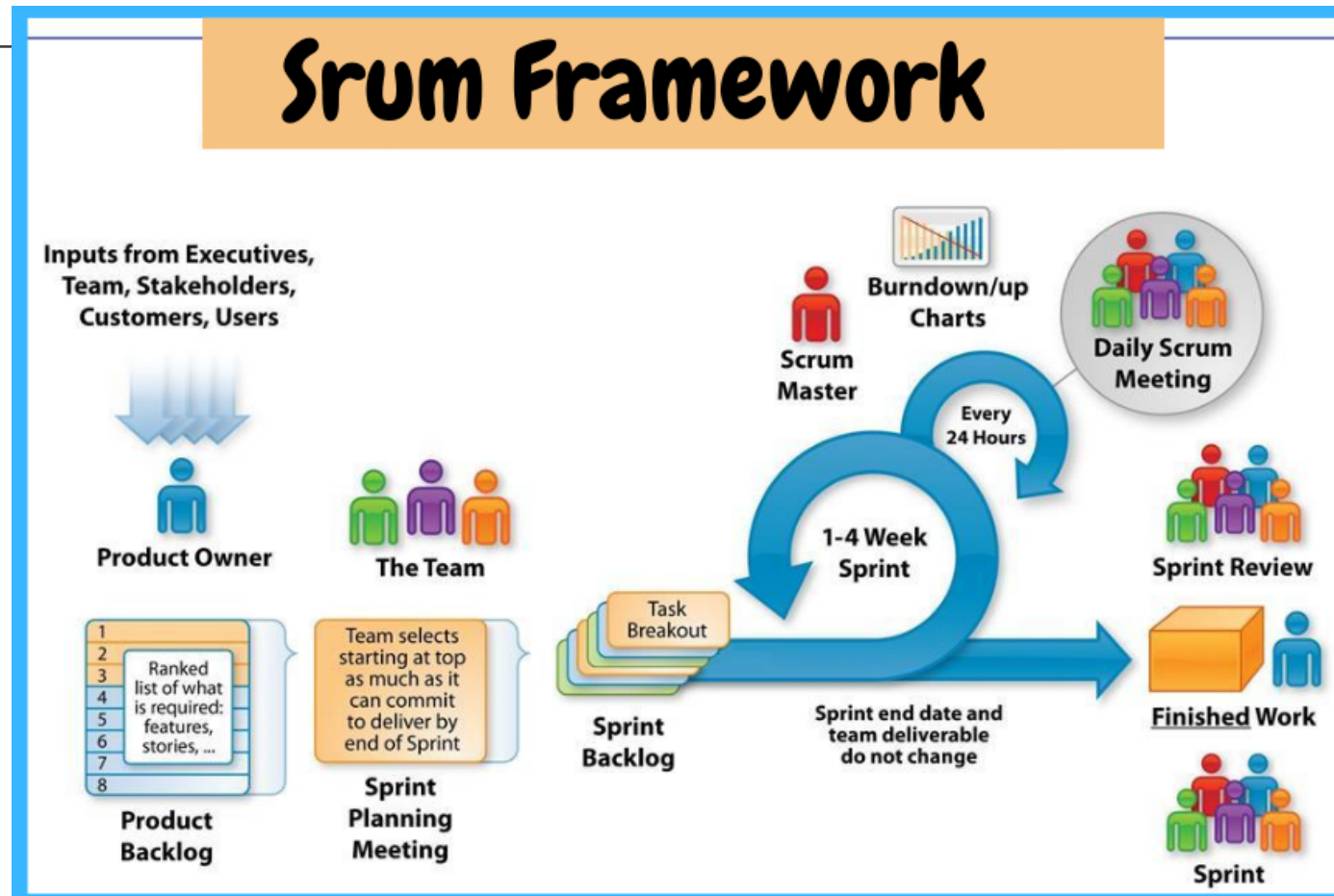
eXtreme Programming [1 of 2]

- **Short, incremental development cycles**
- **Focus on automated tests written by programmers**
- **Emphasis on two-person programming teams**
- **Customers to monitor the development process**
- **Relevant parts of eXtreme Programming that relate to design specifications are**
 - 1. How planning, analysis, design, and construction are all fused into a single phase of activity**
 - 2. Its unique way of capturing and presenting system requirement and design specifications**

eXtreme Programming [2 of 2]

- **Coding and testing are related parts of the same process**
- **Advantages include**
 - **Increased communications among developers**
 - **Higher levels of productivity**
 - **Higher quality code**
 - **Reinforcement of other practices in eXtreme Programming**
 - **Include code-and-test discipline**

Figure 9: Scrum



Originated in 1995 by Sutherland and Schwaber

Most popular methodology for agile (58%)

- **Scrum framework includes**
- **Scrum teams with associated roles, events, artifacts, and rules**
- **Each team consists of three roles**
 - **Product owner**
 - **Development team**
 - **Scrum master**

- **Scrum designed for speed and multiple functional product releases**
- **Primary unit is the Sprint (runs two weeks to a month)**
 - **Starts with an eight-hour planning meeting**
 - **What needs to be delivered by the end of the sprint**
 - **How will the team accomplish that work**
 - **Daily Standup: A 15-minute meeting held to evaluate progress made within the past 24 hours and what needs to be done**

- **At the end of the sprint, two additional meetings**
 - **The Sprint Review: (4 hours) focusing on the product, what has been accomplished, and what needs to be done**
 - **The Sprint Retrospective: (3 hours) focusing on team performance and how it can improve**
- **Three primary artifacts in the Scrum process**
 - 1. Product Backlog: Listing of potential requirements**
 - 2. Sprint Backlog: Listing of only items to be addressed in a particular sprint**
 - 3. Increment: Represents the sum of all the Product Backlog items completed during a sprint.**

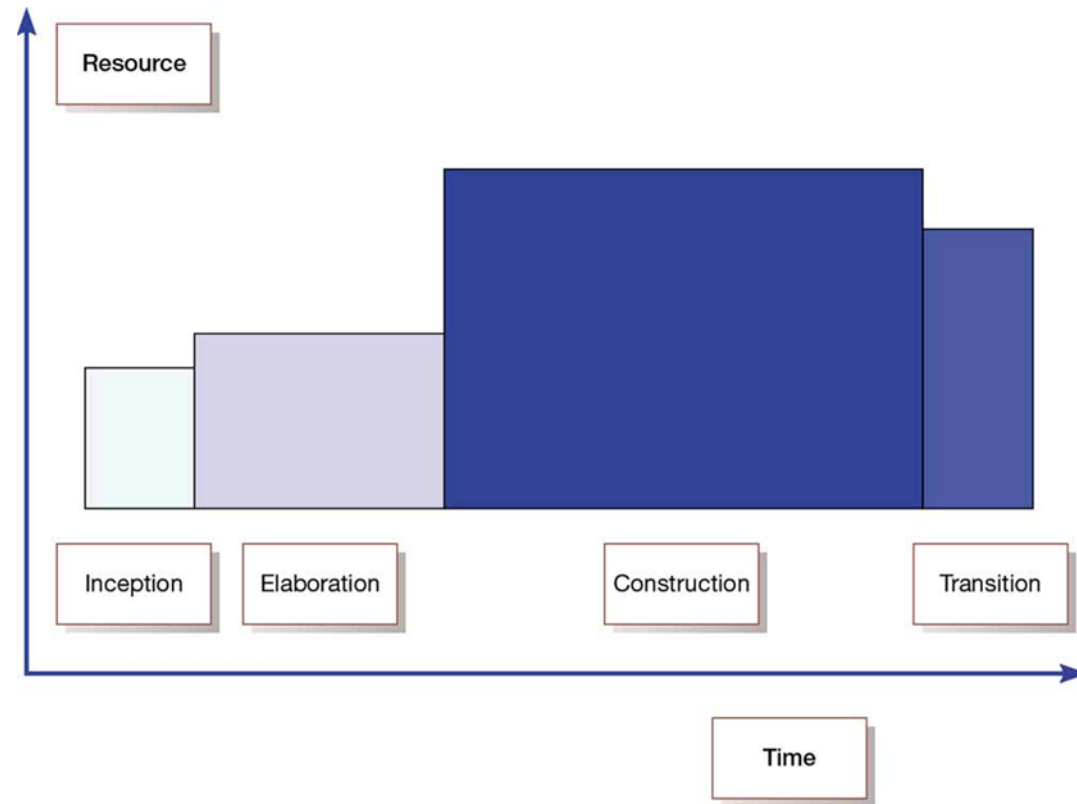
Object-Oriented Analysis and Design [OOAD]

- **Based on objects rather than data or processes**
- **Combines data and processes (called methods) into single entities called objects**
- **Object: A structure that encapsulates attributes and methods that operate on those attributes**
- **Inheritance: Hierarchical arrangement of classes enabling subclasses to inherit properties of superclasses**
- **Object Class: Logical grouping of objects that have the same attributes and behaviors**

Relational Unified Process [RUP]

- **Relational Unified Process (RUP) is an object-oriented systems development methodology**
- **Based on an iterative, incremental approach to systems development**
- **RUPs four phases (each can be further divided)**
 - **Inception**
 - **Elaboration**
 - **Construction**
 - **Transition**

Figure 1-9: Phases of OOAD-Based Development



Roles of an Information System Engineer

....not a job title, but a role that people play:



Requirements owner



System designer



System analyst



Verification & Validation



Logistics & Operation



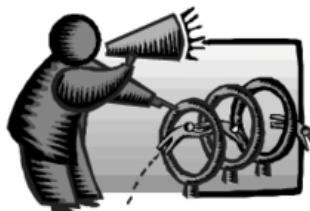
Glue engineer



Customer interface



Co-ordinator



Technical manager



Information manager



Process engineer



Classified ads engineer

Summary

- Define Information Systems Engineering
- Describe the information systems development life cycle (SDLC)
- Describe the agile methodologies, eXtreme Programming, Scrum and RUP
- Describe the roles of an information system engineer